

스마트 배터리 관리

1조

20225099 김도균

20211449 이홍

20225115 임유탉

20225118 최주영

CONTENTS

01

설계주제 개요

02

설계작품 개요도

03

설계 작품의 특징

04

시스템 동작 흐름

05

주요 기술

06

최종 결과

07

활용 방안 및 기대 효과

스마트 배터리 관리

설계 주제 개요

“라즈베리 파이를 이용한 스마트 에너지 관리 프로젝트”

AI (인공지능) + IoT (센서) + Electrification (전기화) + Renewable/Sustainable (친환경)

스마트 배터리 관리

설계 주제 개요

스마트 배터리 관리

- ✓ AI 기반 알고리즘
- ✓ 충전 상태 실시간 분석
- ✓ 자동으로 충전 제어

목표

배터리 수명 연장
에너지 낭비 감소
사용자의 충전 편의성과 안전 향상

스마트 배터리 관리

설계작품 필요성

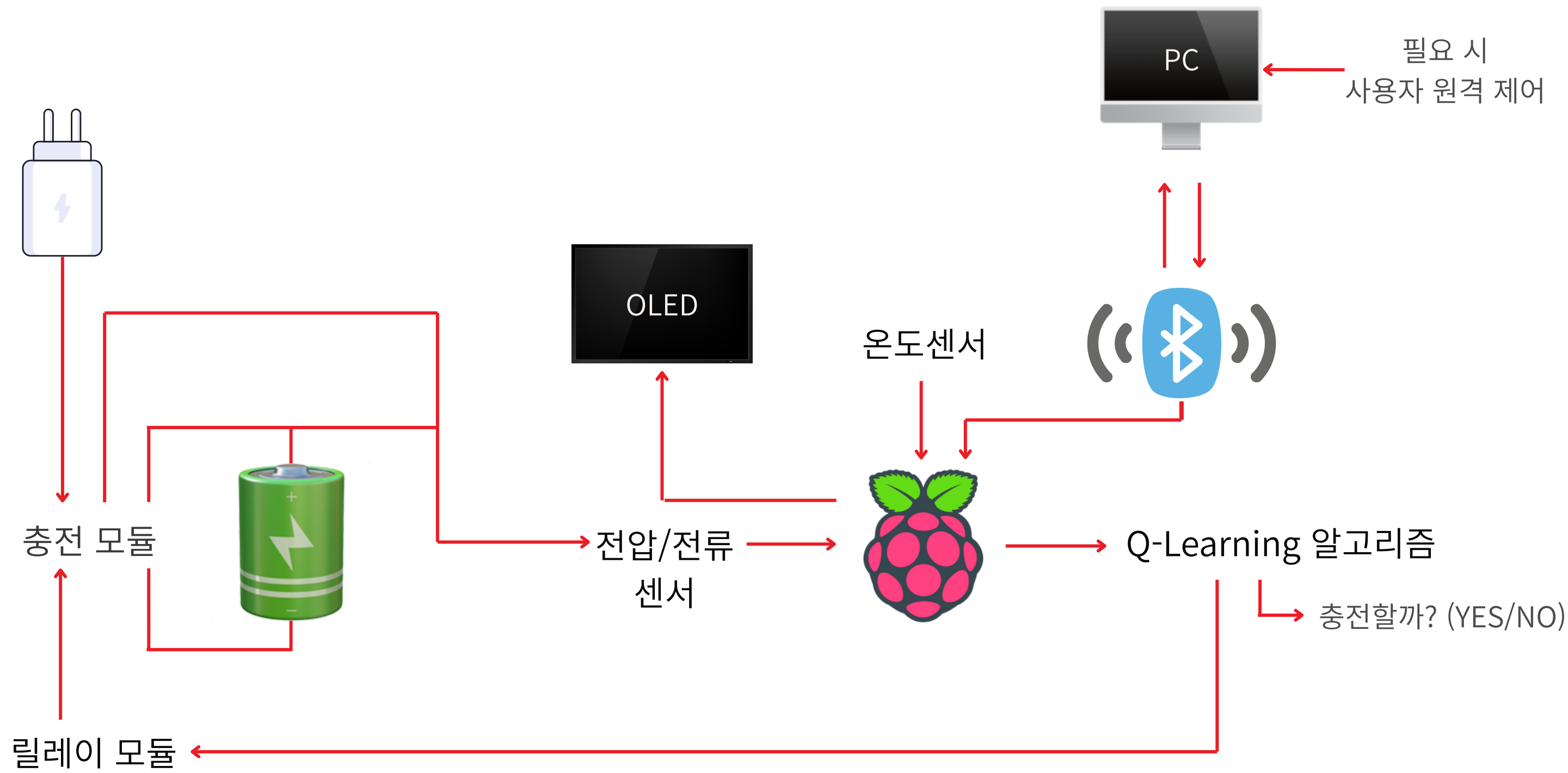
과충전
불필요한 전력 소비
배터리의 수명에 영향



AI 통한 충전 패턴 분석
충전 자동 제어 설계
사용자에게 현재의 충전 상태 보고

배터리 상태를 스마트하게 관리하여 배터리 수명 아끼고 불필요한 전력 소비 감소

스마트 배터리 관리
설계작품 개요도



설계 작품의 주요 특징

데이터 센싱

온도 센서 → 배터리 온도 측정
전류 센서 → 전류 및 전압 상태 측정

인공지능

Q-Learning 기반 강화학습
최적 충전 여부 판단
효율적인 충전 결과

데이터 관리

충전 기록, 온도 변화, 전류 및 전압 상태 관리
전력 소비량 및 충전 시간 예측을 위한 데이터 누적

Actuator 제어

릴레이 모듈로 전자식 충전 제어
OLED를 통한 인터페이스 출력

스마트 배터리 관리

시스템 동작 흐름

01

실시간 데이터 수집

02

상태 계산 및 추정

03

AI 기반 충전 제어

04

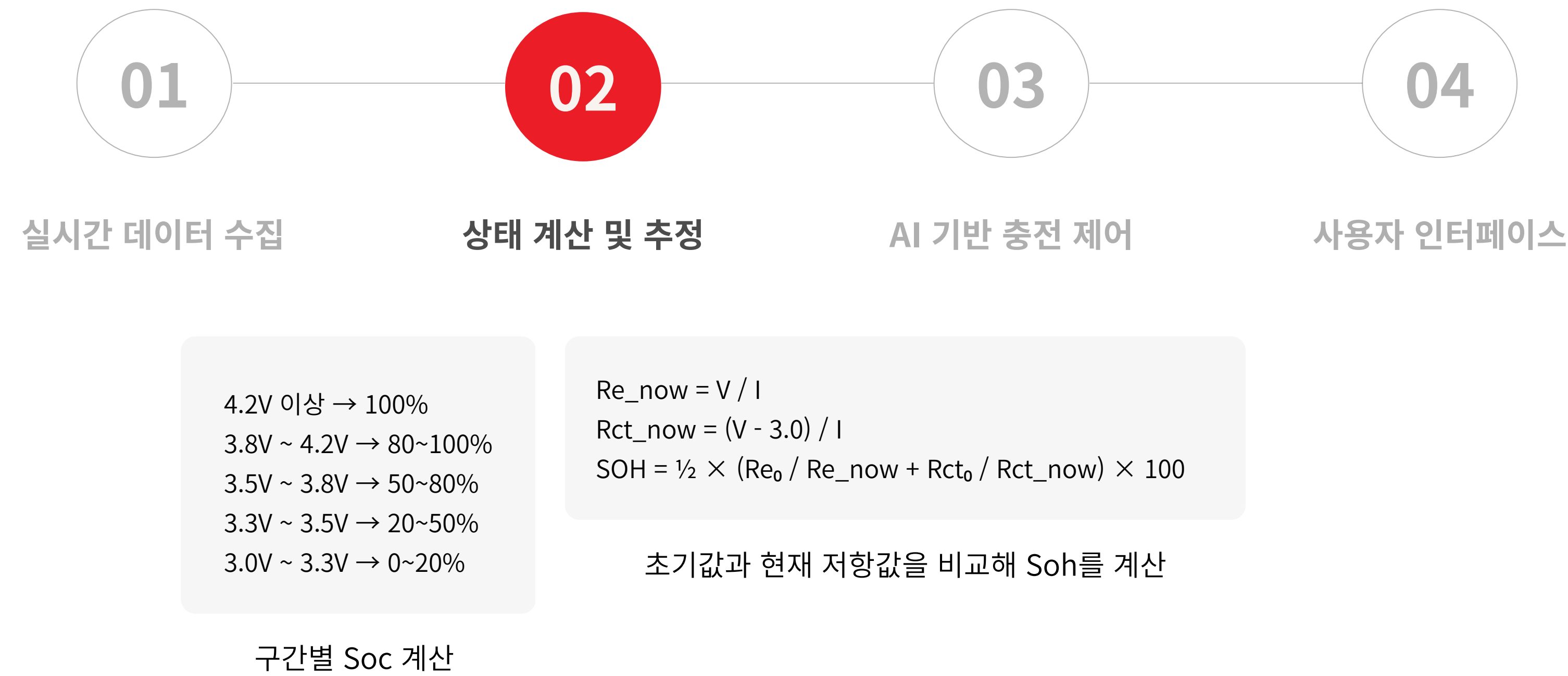
사용자 인터페이스

```
voltage = bus_voltage + shunt_voltage  
current = ina219.current  
temp_c = float(lines[1].split('t=')[1]) / 1000.0
```

센서값을 바탕으로 전류, 전압, 온도 값 추출
센서가 저장한 데이터를 가공하여 °C 단위 온도 변환

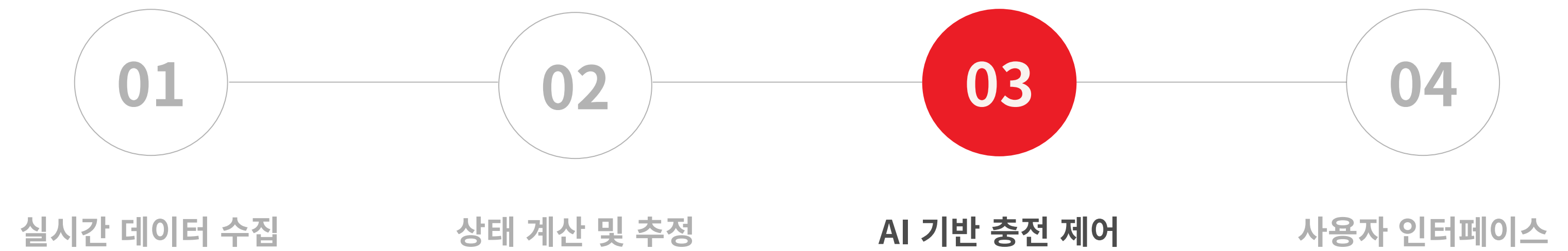
스마트 배터리 관리

시스템 동작 흐름



스마트 배터리 관리

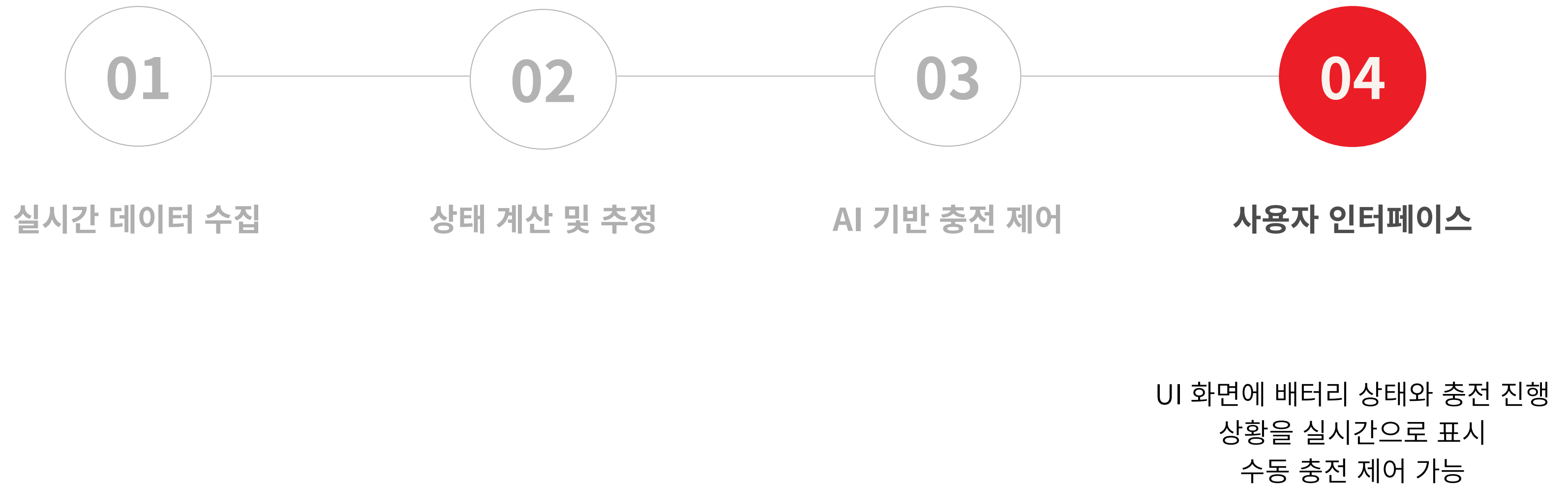
시스템 동작 흐름



온도 45 C 이상
전압 4.26 V 이상 → 충전 중단, 경고 알림
전류 3.5 A 이상

배터리 25% 이하 → 정상 충전
배터리 80% 이상 → 느리게 충전

스마트 배터리 관리 시스템 동작 흐름



주요 기술 : SoC

SoC 측정

전압 기반으로 배터리의 잔량을 추정

구간별 Soc 계산

SOC 증가량 / 전압차이

$$SOC(V) = \begin{cases} 100, & V \geq 4.2 \\ 80 + (V - 3.8) \times 20/0.4, & 3.8 \leq V < 4.2 \\ 50 + (V - 3.5) \times 30/0.3, & 3.5 \leq V < 3.8 \\ 20 + (V - 3.3) \times 30/0.2, & 3.3 \leq V < 3.5 \\ \max(0, (V - 3.0) \times 20/0.3), & V < 3.3 \end{cases}$$

4.2V 이상	→	100%
3.8V ~ 4.2V	→	80~100%
3.5V ~ 3.8V	→	50~80%
3.3V ~ 3.5V	→	20~50%
3.0V ~ 3.3V	→	0~20%

스마트 배터리 관리

주요 기술 : SoH

SoH 측정

NASA에서 제공된 1셀에 대한 Li-ion Battery Aging Dataset 참고
→ 센서로부터 측정된 값들과 비교하여 배터리 건강상태 근사 추적

$$C\text{-rate} = \frac{\text{방전 전류}}{\text{정격 용량}} \quad \frac{5A}{3.45Ah} \approx 1.45C$$

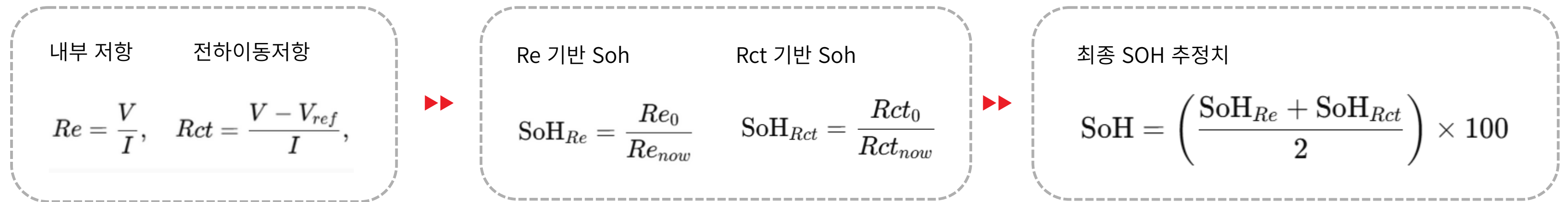
배터리 사양 비교

	(프로젝트) 리튬이온 배터리	NASA Dataset
형태	18650	18650
정격 전압	3.6V	3.6V
정격 용량	3450mAh	2000mAh
테스트 온도	24~43 °C	24~43 °C
충전 전압	4.2V	4.2V
방전	고율 5A (1.45C) 일반 680mA (0.2C) (일반은 비교 대상x)	고율 2A (1.8C)

스마트 배터리 관리

주요 기술 : SoH

리튬 이온 배터리 내부저항 직접 측정 불가하여 측정한 전압·전류 값으로 근사 대체하여 설정



▶▶ smoothing 기법으로 오차 최소화

주요 기술 : Q-learning

Q-Learning

AI 강화학습 기반 충전 최적화

에이전트가 전류, 전압, 온도, 건강상태(soh)를 실시간으로 학습
→ 가장 최적의 조건을 찾아 충전을 지능적으로 제어

벨만 방정식 기반 학습 (Q함수)

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

s	현재 측정되는 전류, 전압, 온도 및 현재 배터리 건강상태
a	s의 상태에서 가능한 행동 (충전 시작 혹은 멈춤 (릴레이 ON/OFF))
r	받은 보상
s'	다음으로 측정되는 전류, 전압, 온도 및 현재 배터리 건강상태
a'	s'의 상태에서 가능한 행동 (충전 시작 혹은 멈춤 (릴레이 ON/OFF))
α	에이전트의 학습률
γ	미래 상태에서의 최대 기대 보상을 현재로 가져오기 위한 할인율 (본 프로젝트는 짧고 즉각적인 보상이 많은 환경에 사용되므로 감마가 0.9 미만)

주요 기술 : Q-learning

DQN (Deep Q-Network)

Q-table 메모리의 한계에 대한 대안으로 DQN 모델 사용
경험 리플레이 + TD Error 기반 점진적 오차 감소 학습

- TD Error : 에이전트가 예측한 Q값과, 실제로 관측한 보상 기반 Q값의 차이
$$\delta = r + \gamma \cdot \underset{\text{Target Q-value}}{a' \max Q_{\text{target}}(s', a')} - \underset{\text{현재 Q값}}{Q_{\text{policy}}(s, a)}$$

→ 벨만 방정식과 매우 유사

딥러닝 모델 → Q(s,a) 예측 근사화 및 업데이트



경험 저장 → 랜덤 샘플링으로 학습 안정화



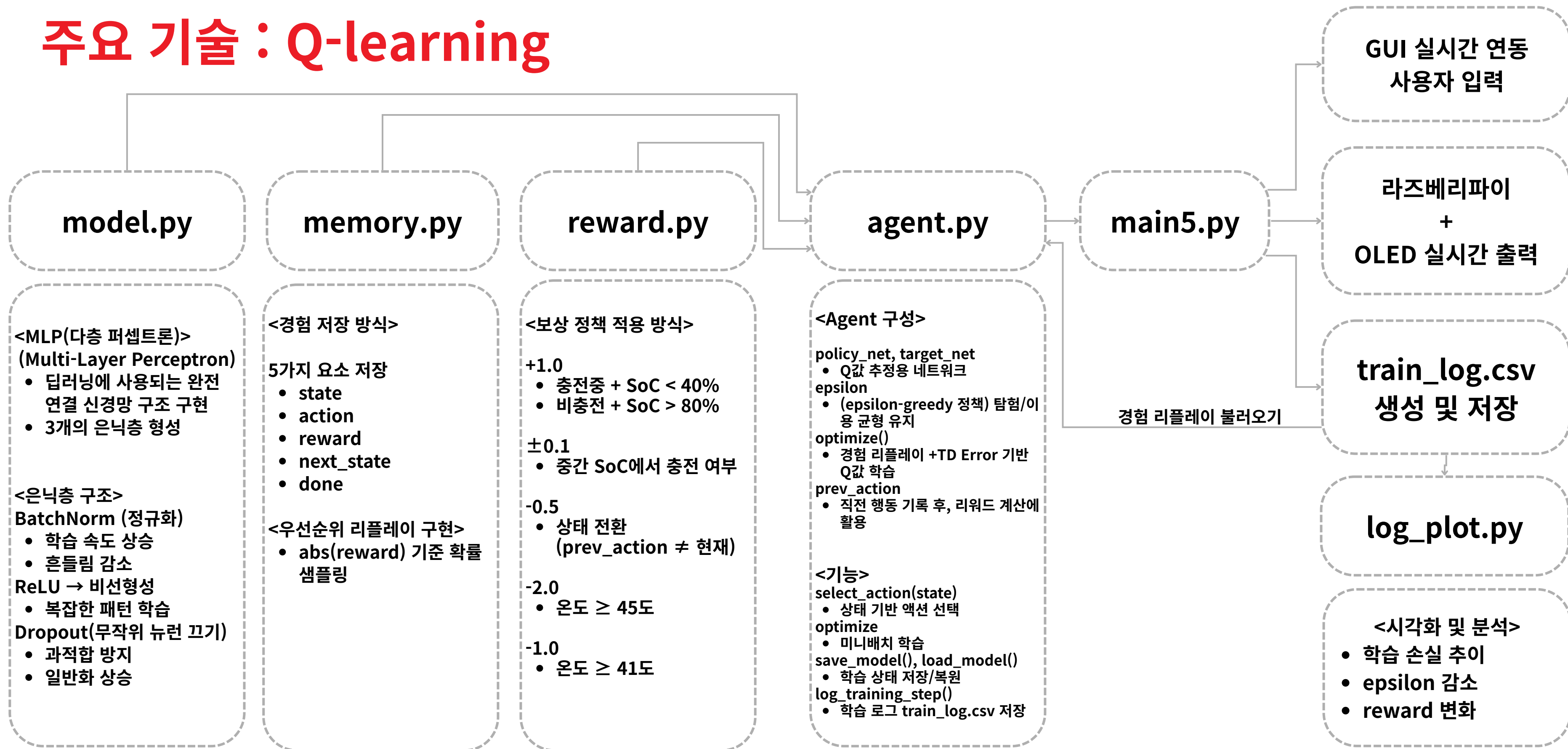
예측 네트워크와 별도로 타깃 네트워크 사용 → 안정성 향상



ϵ -Greedy 정책 → 최적 행동 선택

스마트 배터리 관리

주요 기술 : Q-learning



스마트 배터리 관리

최종 결과 : 하드웨어

전류센서모듈

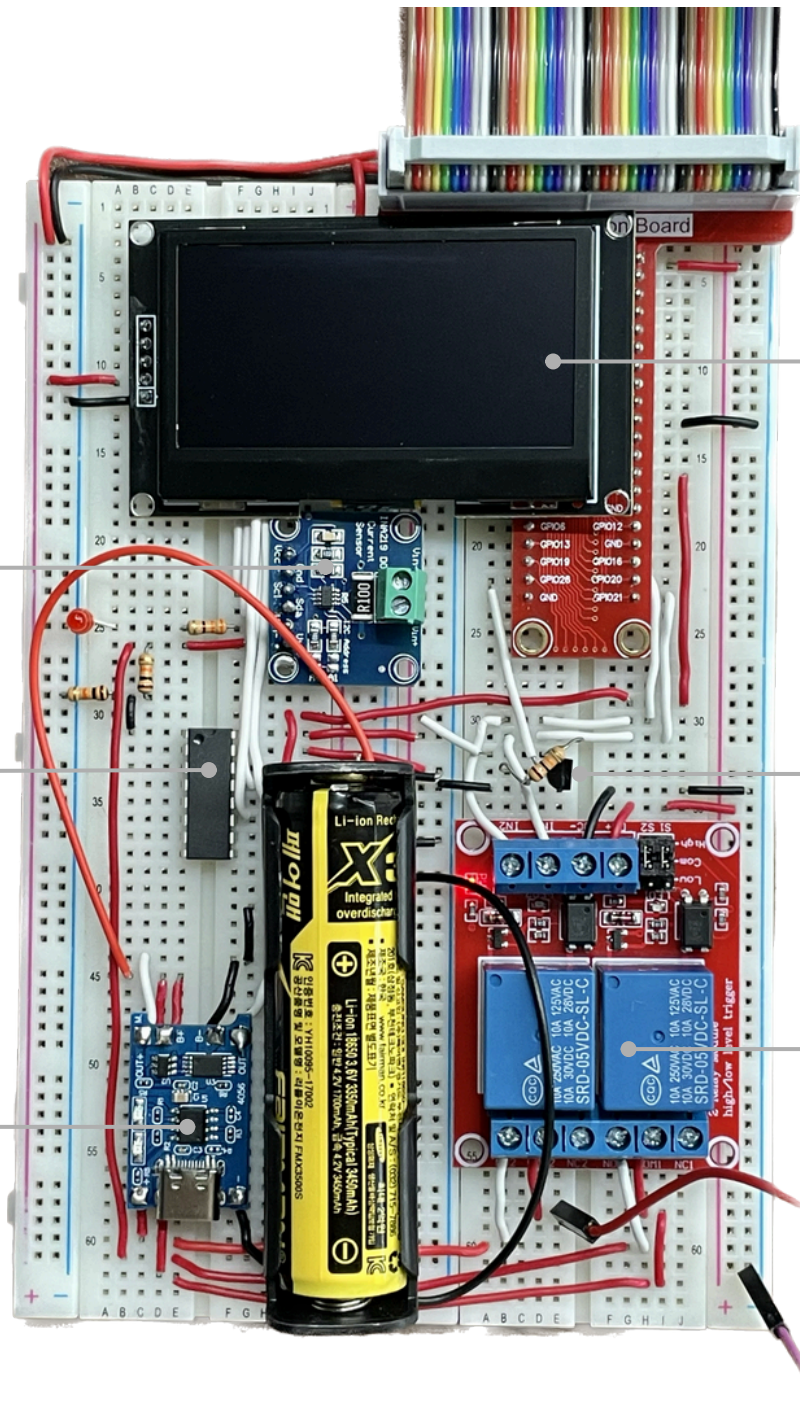
배터리 전류와 전압 실시간으로 측정
라즈베리파이와 I2C 통신으로 데이터를
전송하여 충전 상태 판단에 활용

디지털 컨버터 (ADC)

아날로그 센서 출력을 디지털 신호로 변환

충전모듈

충전모듈 배터리 연결 포트에 리튬이온배터리 연결
→ 출력포트로 배터리 전압을 출력
→ 전압분배를 통해 ADC모듈에 아날로그 값 입력



OLED

실시간 전압, 전류, 온도, 배터리 정보 표시
사용자가 시스템 상태 직관적으로 확인

온도센서

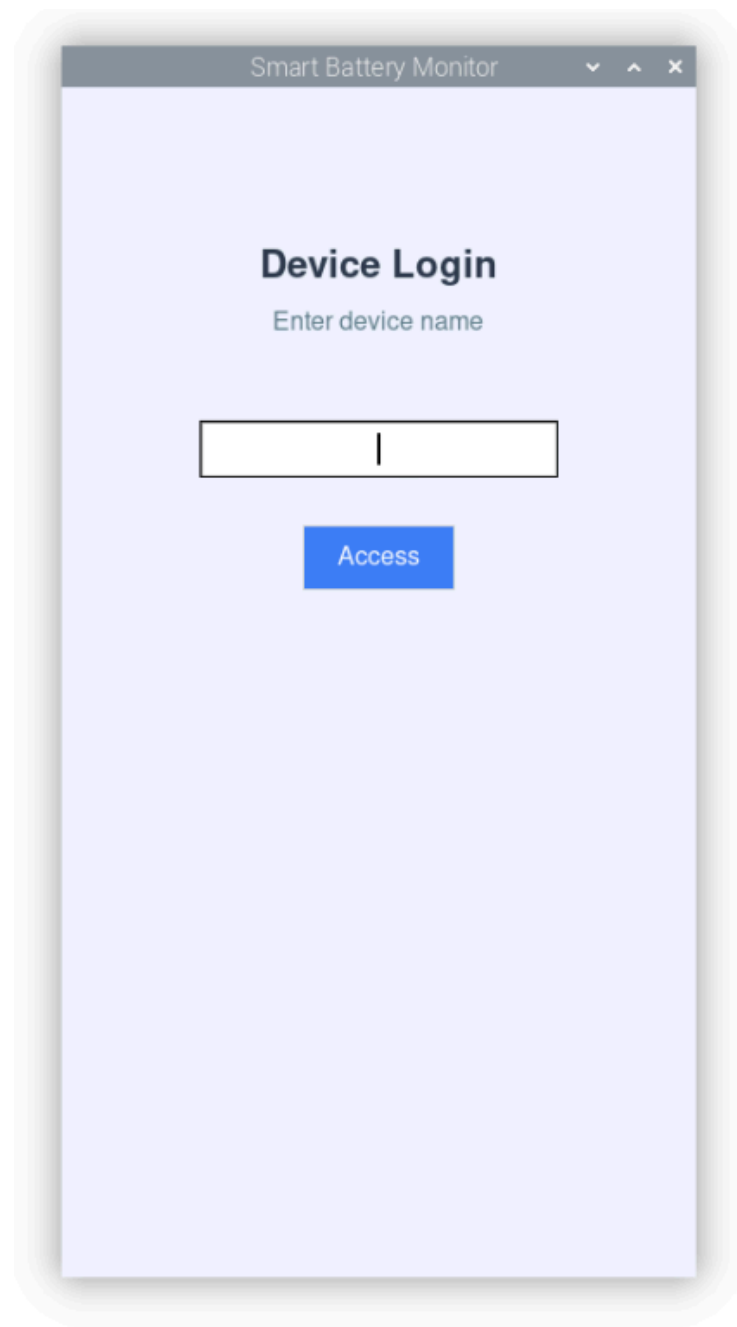
배터리 온도를 측정하여 과열 여부 감지

2-channel relay 모듈

충전 ON → relay1 = ON, relay2 = OFF
충전 OFF → relay1 = OFF, relay2 = OFF

스마트 배터리 관리

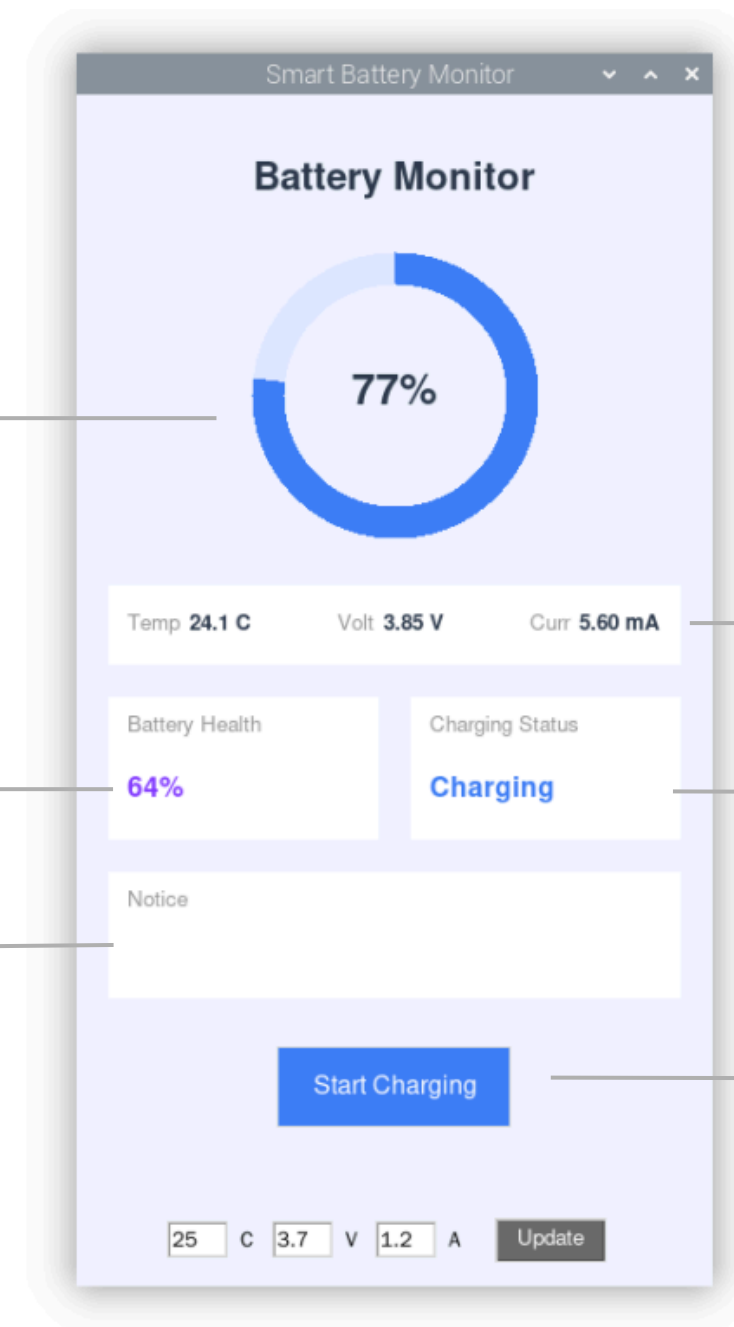
최종 결과 : User Interface



SOC

SOH

알림



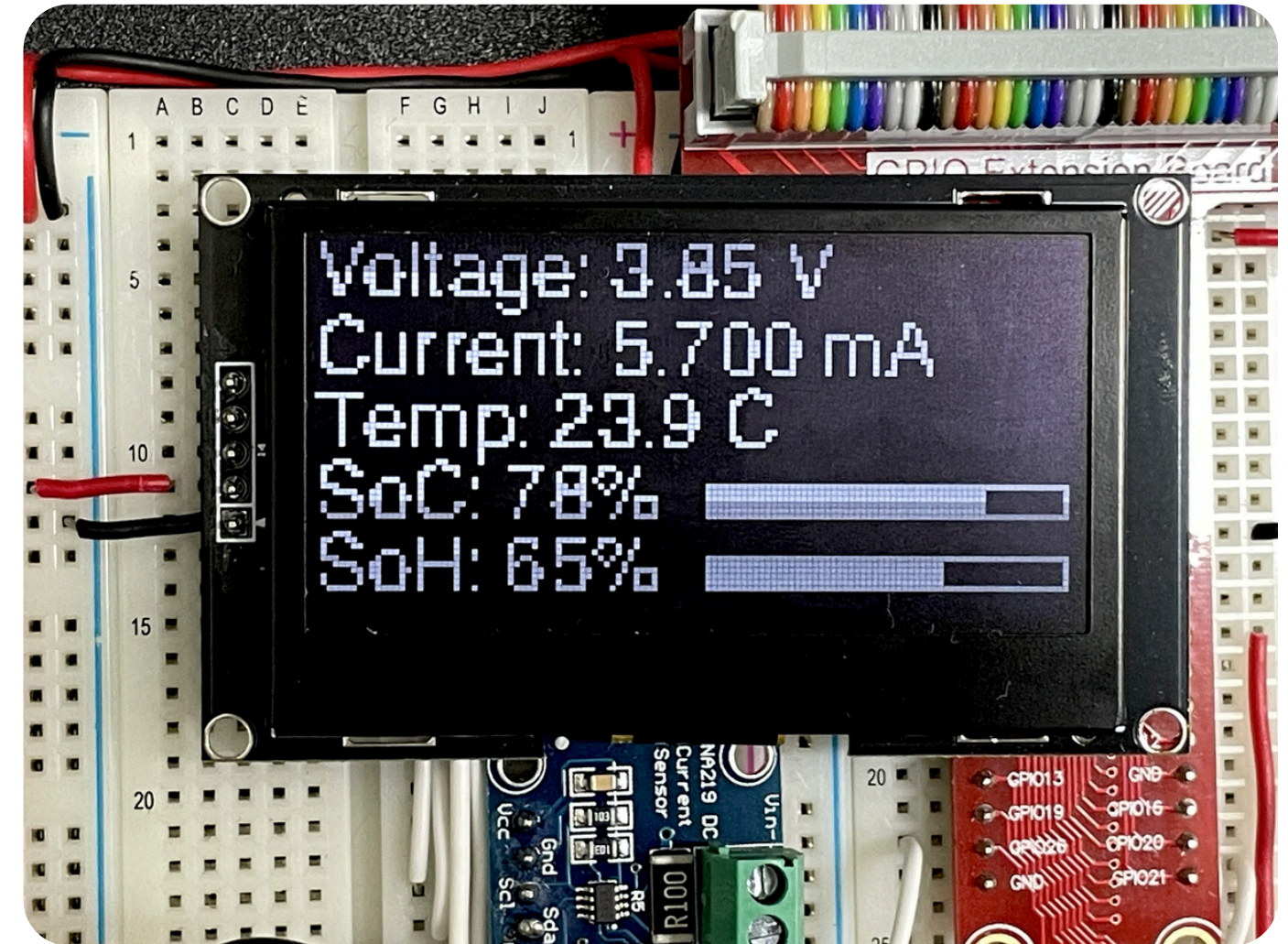
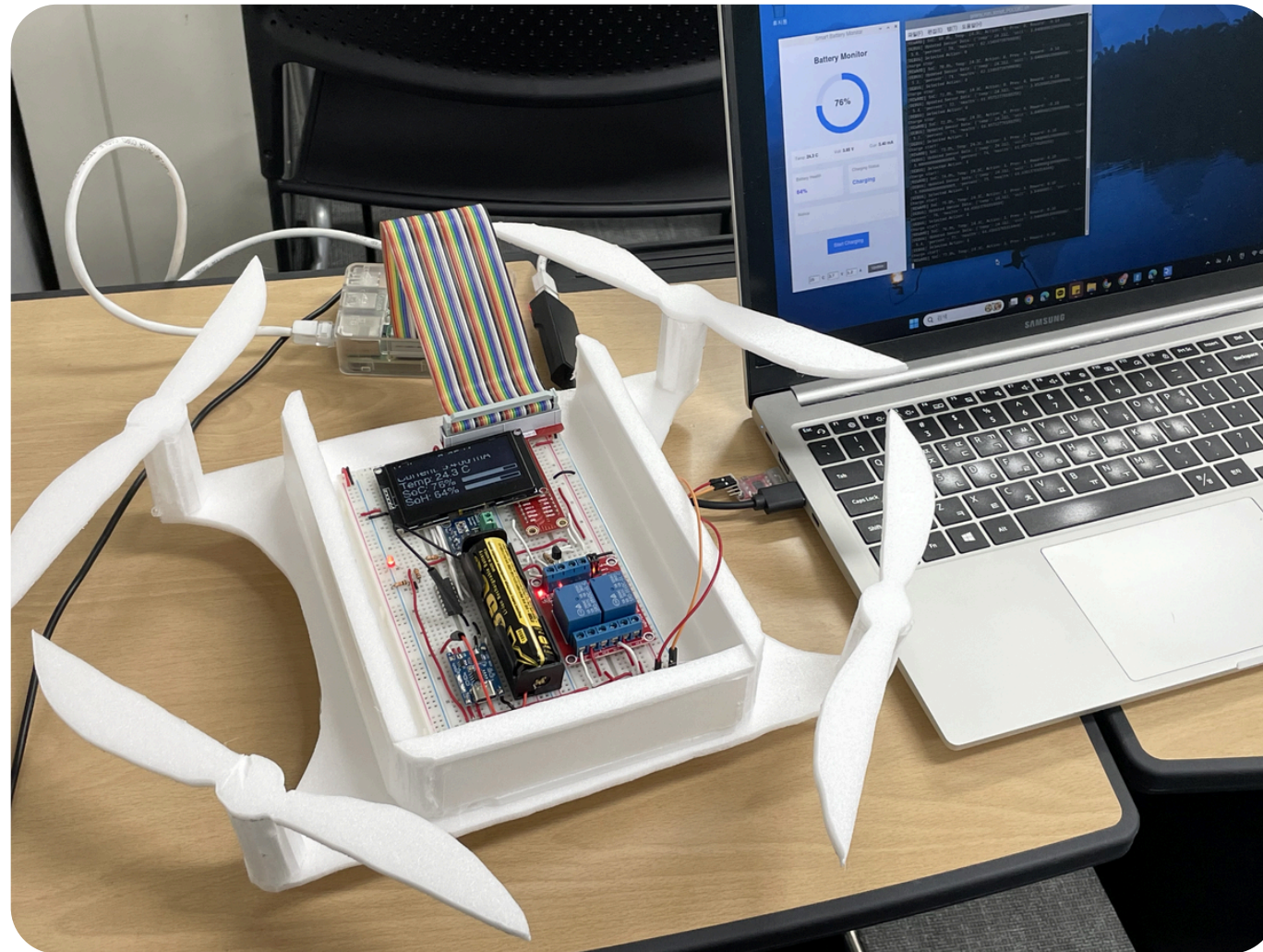
실시간 측정값

충전 ON/OFF 상태

충전 수동제어

스마트 배터리 관리

최종 결과



스마트 배터리 관리

기대 효과 및 활용 방안

드론

완전충전-완전방전 방식
평균 비행 시간이 짧음
수명 짧음
배터리 교체 주기 빠름

전동 킥보드 / 전기 자전거

단순한 배터리 보호회로만 갖추고 있음
완전충전-완전방전 방식
기온에 따라 배터리 성능 저하



기대효과

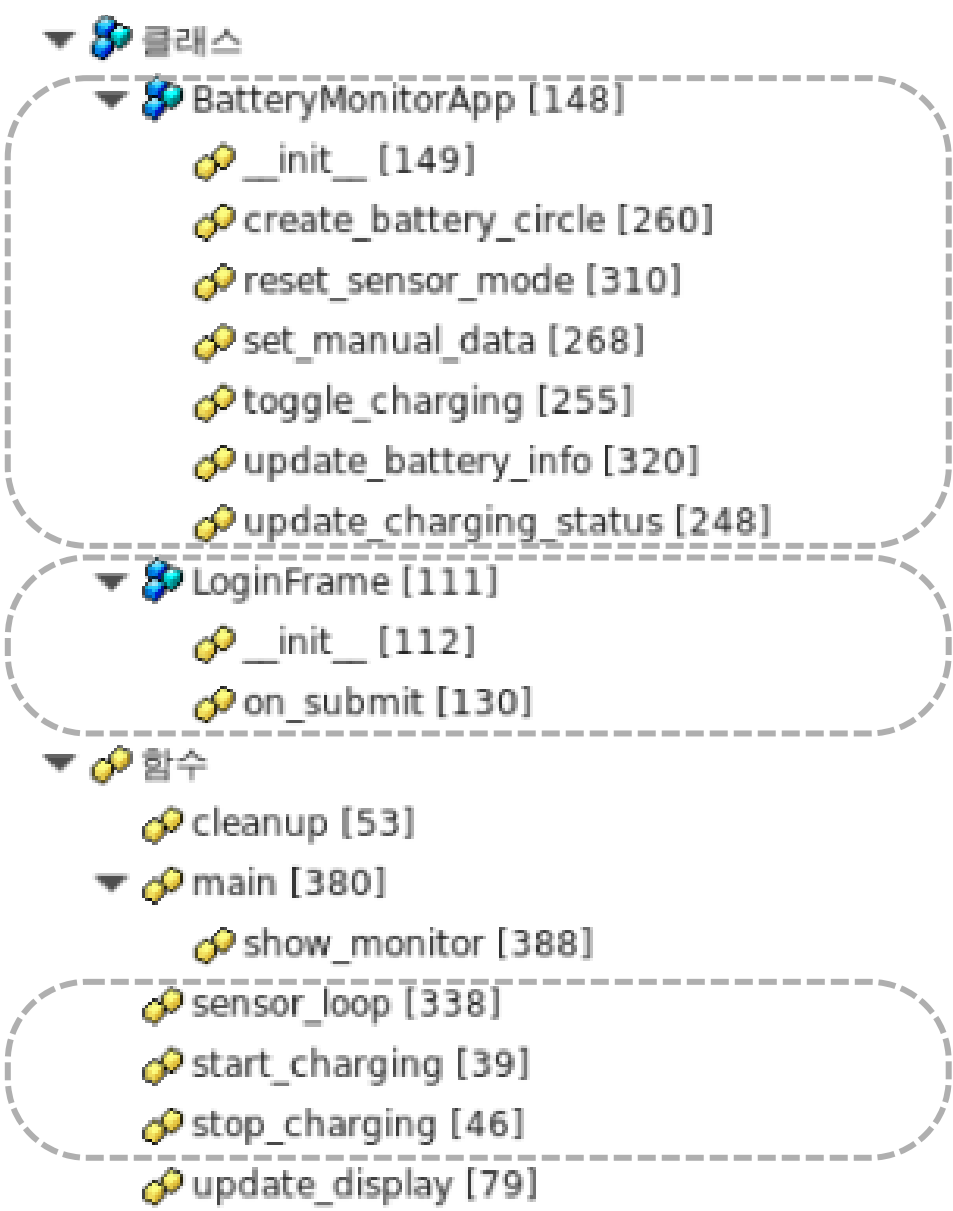
배터리 수명 연장 → 사이클 수 증가
SOC 추정 정확도 향상 → 비행시간, 이동거리 예측 정확도
배터리 상태 시각적 제공 → 사용자 신뢰도 향상

감사합니다

Appendix

프로그램 코드 설명

main5.py



GUI
모니터링 화면

기기 로그인 화면

충전 제어 로직
OLED 디스플레이 출력

Appendix

프로그램 코드 설명

register_device.py

데이터베이스를 이용해 사용자로부터 입력받은 기기명을 devices 테이블에 등록

```
import sqlite3

conn = sqlite3.connect("devices.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS devices (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT UNIQUE NOT NULL
)
""")

device_name = input("등록할 기기명을 입력하세요: ").strip()

try:
    cursor.execute("INSERT INTO devices (name) VALUES (?)", (device_name,))
    conn.commit()
    print(f"'{device_name}' 기기가 등록되었습니다.")
except sqlite3.IntegrityError:
    print("이미 등록된 기기입니다.")

conn.close()
```

Appendix

프로그램 코드 설명

sensor_module.py

INA219 + DS18B20 측정, Soc, SoH 계산 및 필터링

- ▼ 클래스
 - ▼ SOHFilter [93]
 - __init__ [94]
 - update [99]
- ▼ 함수
 - estimate_battery_percentage [73]
 - get_sensor_data [168]
 - initialize_ina219 [35]
 - read_sensor_data [143]
 - read_temp_c [60]
 - read_temp_raw [56]
 - run_realtime_soh_monitoring [110]
 - save_sensor_data_to_csv [86]
 - update_battery_percentage [47]
- ▶ 변수

relay_module.py

RELAY_IN1, RELAY_IN2 두 핀으로 충전 흐름 제어

- ▼ 함수
 - battery_only [24]
 - cleanup [30]
 - start_charging [14]
 - stop_charging [19]
- ▼ 변수
 - RELAY_IN1 [4]
 - RELAY_IN2 [5]
 - cmd [39]

Appendix

프로그램 코드 설명



dqn



__pycache__



agent.py



memory.py



model.py



reward.py

Appendix

프로그램 코드 설명



dqn

agent.py

policy_net, target_net : Q값 추정용 네트워크
epsilon : (epsilon-greedy 정책) 탐험/이용 균형 유지
prev_action : 직전 행동 기록 후, 리워드 계산에 활용

- 상태 기반 액션 선택
- 경험 리플레이 +TD Error 기반 Q값 학습
- 학습 상태 저장/복원
- 학습 로그 train_log.csv 저장

```
import torch
import torch.optim as optim
import torch.nn.functional as F
import numpy as np
import csv
import os
import sys
sys.path.append("/home/pi/workspace/project/real/test")
from dqn.model import DQN
from dqn.memory import ReplayMemory
import random

class Agent:
    def __init__(self, state_dim, action_dim, device):

    def log_sensor_data(self, state):

    def select_action(self, state):

    def optimize(self):

    def load_model(self, path="dqn_model.pth"):#
    def save_model(self, path="dqn_model.pth"):#
    def log_training_step(self, step, loss, epsilon, reward, action, log_path="train_log.csv"):
```

Appendix

프로그램 코드 설명



dqn

model.py

MLP(다층 퍼셉트론)

(Multi-Layer Perceptron)

딥러닝에 사용되는 완전 연결 신경망 구조 구현
3개의 은닉층 형성

은닉층 구조

BatchNorm (정규화)

학습 속도 상승

흔들림 감소

ReLU → 비선형성

복잡한 패턴 학습

Dropout(무작위 뉴런 끄기)

과적합 방지

일반화 상승

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DQN(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(DQN, self).__init__()
        self.fc1 = nn.Linear(input_dim, 256) # Increased neurons for deeper learning
        self.bn1 = nn.BatchNorm1d(256) # Batch Normalization for stable training
        self.fc2 = nn.Linear(256, 128)
        self.bn2 = nn.BatchNorm1d(128)
        self.fc3 = nn.Linear(128, 128) # Added extra layer for finer adjustments
        self.out = nn.Linear(128, output_dim)
        self.dropout = nn.Dropout(0.3) # Dropout to prevent overfitting

    def forward(self, x):
        x = F.relu(self.bn1(self.fc1(x)))
        x = self.dropout(F.relu(self.bn2(self.fc2(x))))
        x = F.relu(self.fc3(x))
        #return torch.sigmoid(self.out(x)) # Activation for smoother output
        return self.out(x)
```


Appendix

프로그램 코드 설명



dqn

memory.py

경험 저장 방식

→ 5가지 요소 저장

state
action
reward
next_state
done

우선순위 리플레이 구현

abs(reward) 기준 확률 샘플링

```
import random
import numpy as np
from collections import deque

class ReplayMemory:
    def __init__(self, capacity):
        """Initialize Replay Memory with prioritization."""
        self.memory = deque(maxlen=capacity)
        self.priorities = deque(maxlen=capacity) # Track experience priorities

    def push(self, state, action, reward, next_state, done):
        """Store experience with its priority."""
        experience = (state, action, reward, next_state, done)
        self.memory.append(experience)

        # Assign priority based on absolute reward (higher reward -> higher priority)
        priority = abs(reward) + 1e-5 # Small offset to prevent zero priority
        self.priorities.append(priority)

    def sample(self, batch_size):
        """Sample experiences based on priority."""
        if len(self.memory) < batch_size:
            return random.sample(self.memory, batch_size)

        # Normalize priorities for probability distribution
        priorities = np.array(self.priorities, dtype=np.float32)
        probabilities = priorities / priorities.sum()

        indices = np.random.choice(len(self.memory), batch_size, p=probabilities)
        return [self.memory[i] for i in indices]

    def __len__(self):
        return len(self.memory)
```

Appendix

프로그램 코드 설명



dqn

reward.py

보상	조건
+1.0	충전중 + SoC < 40% 비충전 + SoC > 80%
±0.1	중간 SoC에서 충전 여부
-0.5	상태 전환 (prev_action ≠ 현재)
-2.0	온도 ≥ 45도
-1.0	온도 ≥ 41도

```
def calculate_reward(battery_percent, temperature, is_charging, prev_action):
    reward = 0.0

    # Encourage stable charging behavior
    if is_charging and battery_percent < 40:
        reward += 1.0 # Positive reward for charging when battery is low
    elif not is_charging and battery_percent > 80:
        reward += 1.0 # Positive reward for stopping charge at high battery level
    elif 40 <= battery_percent <= 80:
        reward += 0.1 if is_charging else -0.1

    # Penalize frequent switching of charging state
    if prev_action is not None and prev_action != is_charging:
        reward -= 0.5 # Apply penalty if the state changes too frequently

    if prev_action is None:
        print("[WARN] prev_action is None - using default reward logic")

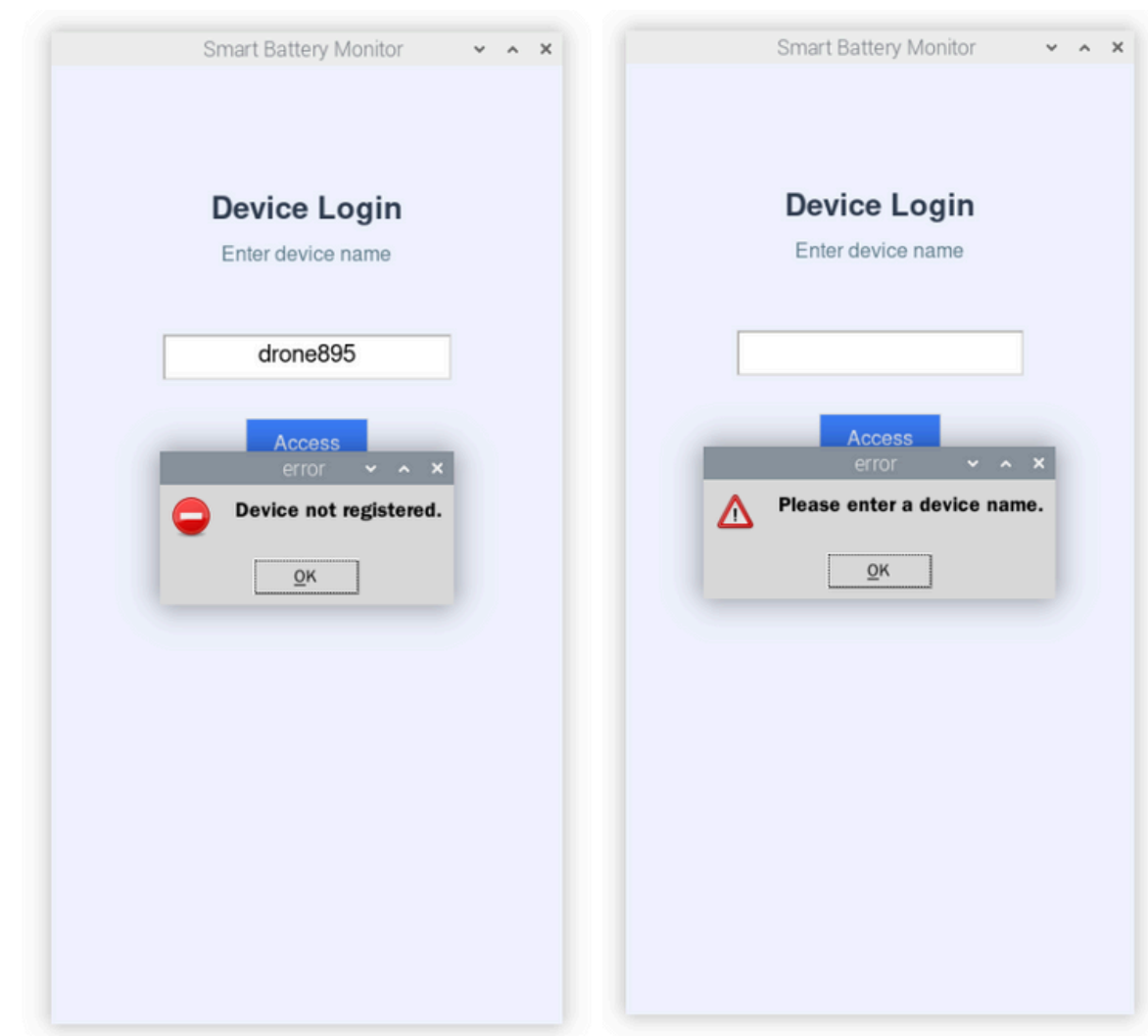
    # Penalize high temperature situations
    if temperature >= 45:
        reward -= 2.0 # Stronger penalty for overheating
    elif temperature >= 41:
        reward -= 1.0
    print(f"[REWARD] SoC: {battery_percent:.1f}%, Temp: {temperature:.1f}C, "
          f"Action: {is_charging}, Prev: {prev_action}, Reward: {reward:.2f}")

    return reward
```

Appendix

UI 추가 설명

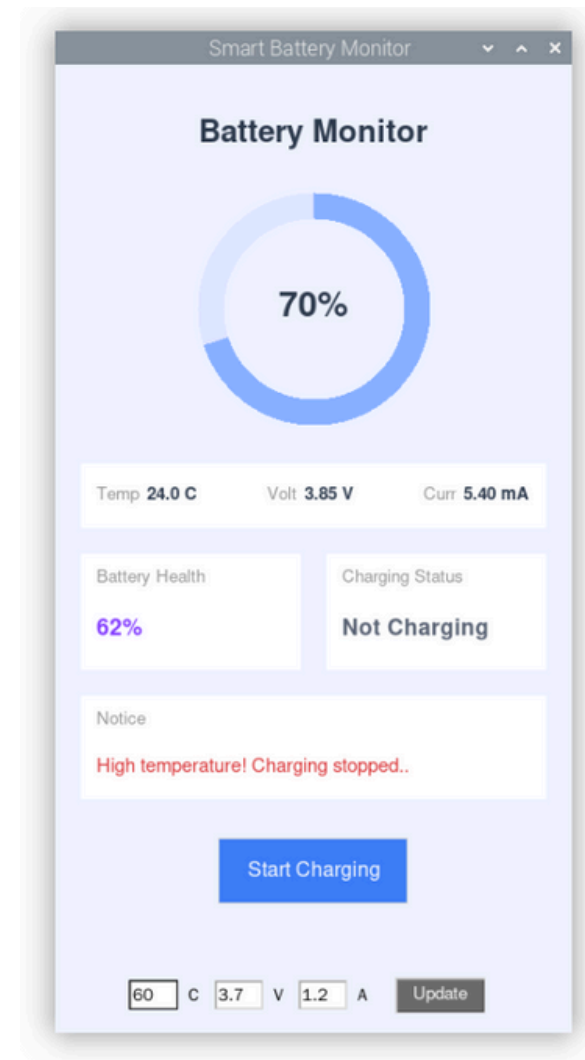
로그인 - 경고창



입력 칸이 비었을 때

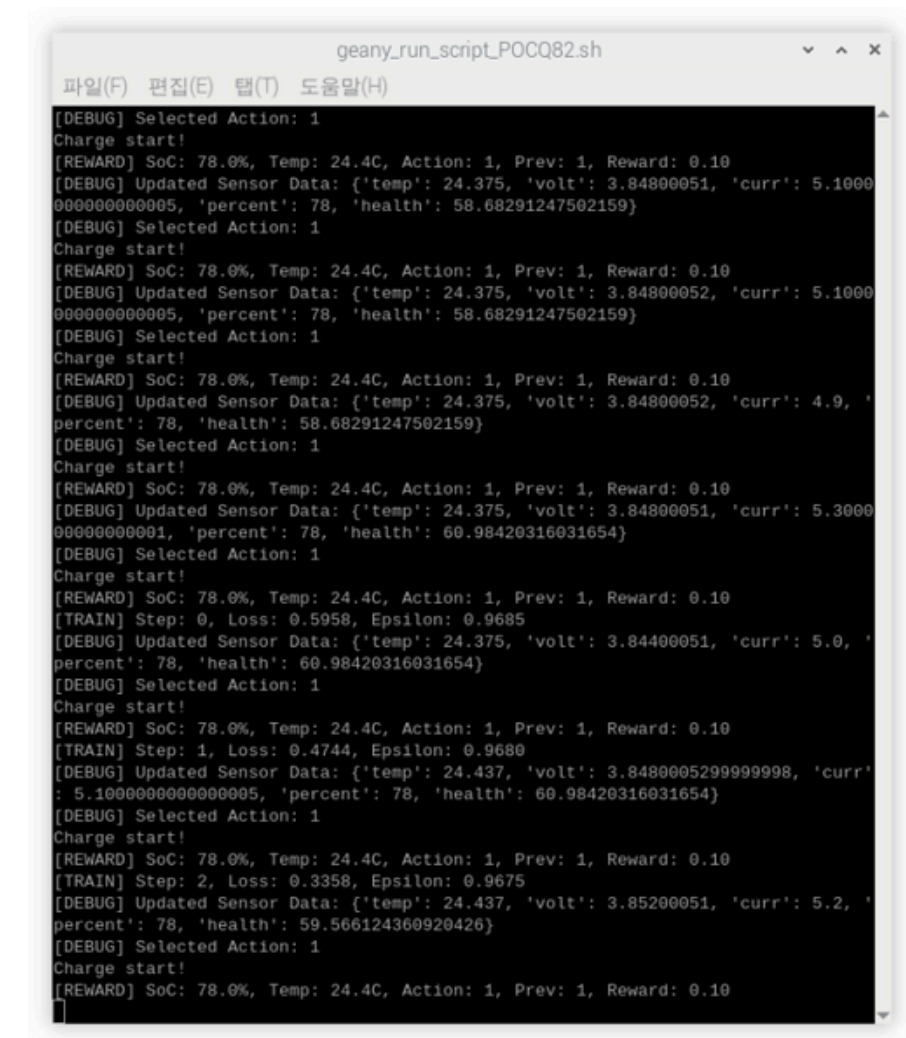
입력한 기기명이 DB에 없을 때

NOTICE



센서 값이 기준을 초과하면
→ Not Charging
→ Notice 상자에 경고 메시지

터미널창



Soc, Soh, Temp, Action, 이전 Action값, 보상값
실시간 업데이트

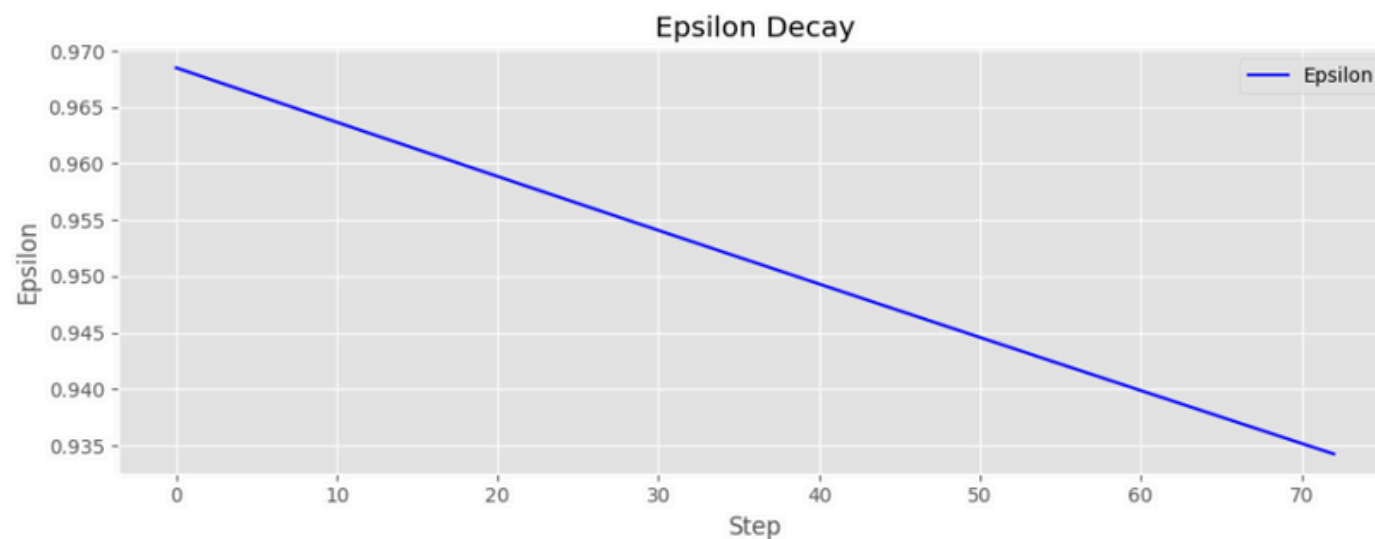
Appendix

DQN 학습 시각화 그래프

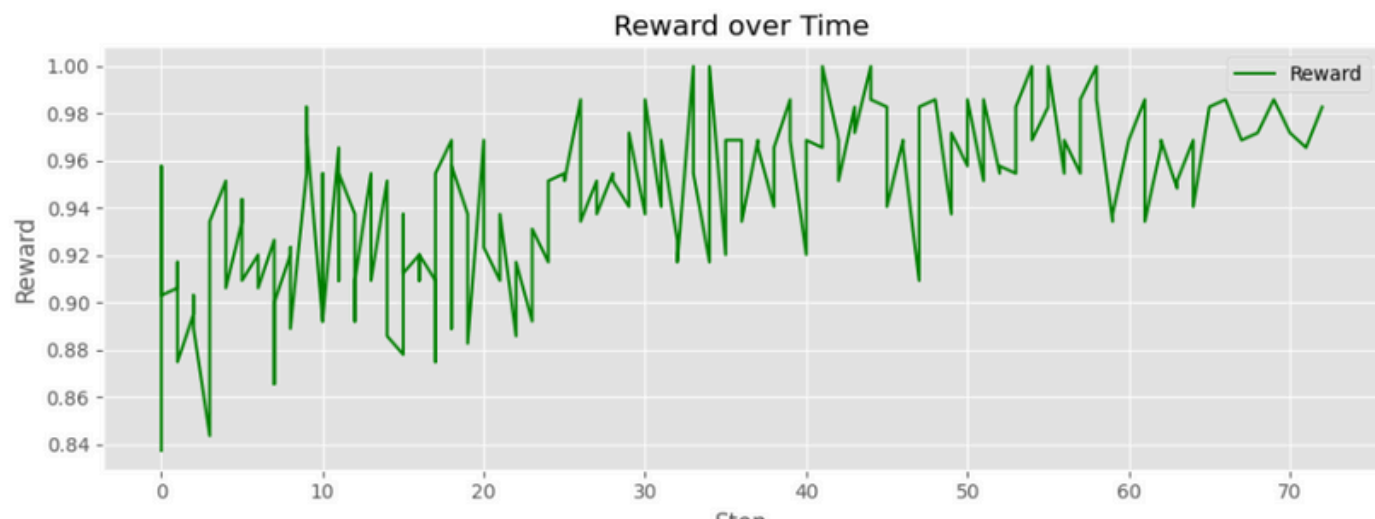
loss_plot.png



epsilon_plot.png



reward_plot.png



loss_plot.png(학습 손실 추이)

매 학습 step마다 계산된 MSE 손실값(TD error의 제곱)

→ 에이전트가 얼마나 정확하게 Q값을 예측하고 있는지 보여주는 그래프

→ 초반에 loss가 높게 시작해서 점차 하강하면 학습이 제대로 이루어졌다는 의미

- loss가 계속 폭등하거나 진동하는 이유는 learning rate 문제와 reward 품질 문제로 인해 발생하는 일시적 증상

epsilon_plot.png(탐험 비율 감소)

ϵ 값이 클수록 무작위 탐험을 많이 함

작을수록 학습된 정책을 따름

→ 1.0에서 시작해서 서서히 감소하면 정책 중심으로 탐험이 진행되고 있다는 의미

→ 최종적으로 0.1~0.05 정도에 수렴해야 함

- 초기 구현에서 경험 리플레이 저장이 제대로 이루어지지 않아 학습이 부족했음. → 현재는 계속 값이 감소하면서 해결됨

reward_plot(리워드 추이)

매 학습 step마다 받는 보상의 평균값

DQN이 상황에 따른 액션을 점점 더 잘 하게 되는지 보여주는 그래프

→ 초반에는 낮거나 흔들림

→ 점점 우상향하여 일정 수준 이상 유지하면 리워드가 제대로 이루어지고 있다는 의미

*step은 상태 → 행동 → 보상 → 다음 상태 전이 과정이 끝날 때마다 증가하는 카운터

*MSE(Mean Squared Error) : TD Error의 제곱, 즉, loss를 의미

→ '오차를 줄이기 위해 어떻게 학습할 것인가'를 결정하는 기준점. 즉, Q값 예측이 실제 측정 Q값과 가까워지게 만들면서 더 나은 정책을 배우는 지표

Appendix

사용한 부품



[SZH-SSBH-075]
INA219 HIGH Side DC 전류 센서 모듈
2000원



[SZH-SSBH-095]
ACS712 5A 전류 센서 모듈
1300원



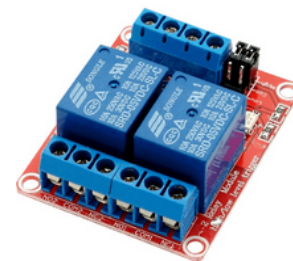
[SZH-BH009]
18650 리튬이온 배터리 홀더 x 1
800원



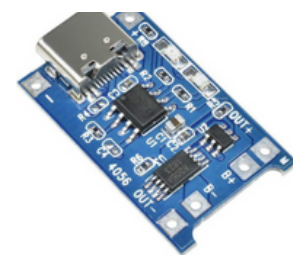
[SZH-BH002]
18650 리튬이온 배터리 홀더 x 2
1980원



[DS18B20]
온도 센서
1800원



[SZH-RLBG-042]
아두이노 2채널 릴레이 모듈 x 2
3520원



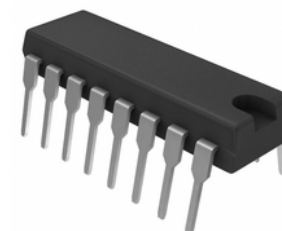
[SZH-LIP001]
TP4056 리튬 배터리 1A 충전 보호회로 USB C-Type
220원



아두이노 2.42인치 I2C OLED 화이트
27500원



18650 리튬 이온 3500mAh 충전용 건전지 x 3
27300원



[MCP3008-I/P]
디지털 컨버터 (ADC)
4200원

Appendix

참고자료

<https://m.blog.naver.com/darknisia/221426129644>

<https://blog.naver.com/eduino/220884752484>

<https://blog.naver.com/sahma/223710728304>

<https://blog.naver.com/behappydw/114551121>

<https://blog.naver.com/happy-player/221751781039>

<https://www.youtube.com/watch?v=IjtJwjvHgNA>

<https://m.blog.naver.com/zeta0807/221592831275>

<https://www.youtube.com/watch?v=jJ-xD8KEhwk>

<https://hardcopyworld.com/spi-oled->

%EB%94%94%EC%8A%A4%ED%94%8C%EB%A0%88%EC%9D%B4-

%EC%82%AC%EC%9A%A9%EB%B0%A9%EB%B2%95/

<https://www.kaggle.com/datasets/patrickfleith/nasa-battery-dataset>